Segment Tree

By the end of this lecture students should be able to:

1. Understand why Segment Tree is required.
2. Identify problems solvable using Segment Tree.
3. Build a Segment Tree.
4. Perform **range queries** and **updates**.
5. Solve typical coding interview problems.

Lecture Structure

Time	Topic
0–10 min	Problem Motivation
10–25 min	Brute Force vs Efficient Approach
25–40 min	Segment Tree Concept
40–60 min	Segment Tree Construction
60–80 min	Range Query
80–95 min	Update Operation
95–110 min	Solving Problems
110–120 min	Identification + Wrap-up

---

1. Motivation

Start with a Problem

Array:

```
[2, 5, 1, 4, 9, 3]
```

Queries:

```
sum(1,4)
sum(0,5)
sum(2,3)
```

Ask students:

How will you solve this?

Students answer:

```
Loop from L to R
```

Time Complexity:

```
O(N)
```

If:

```
N = 10^5
Q = 10^5
```

Total complexity:

```
O(N × Q) = 10^10
```

Explain:

```
Too slow!
```

---

2. Improving the Solution

Prefix Sum Idea

Prefix sum helps if:

```
No updates
```

Example:

```
sum(l,r) = prefix[r] - prefix[l-1]
```

But if update occurs:

```
update index 3 → 7
```

All prefix values must change.

Cost:

```
O(N)
```

Conclusion:

We need a structure that supports:

```
1. Range Query
2. Update
```

Efficiently.

Introduce:

```
Segment Tree
```

---

3. Segment Tree Concept

Explain:

Segment Tree divides array into **segments**.

Example array

```
index: 0 1 2 3 4 5
value: 2 5 1 4 9 3
```

Tree representation:

```
                 [0-5]
              sum=24
           /             \
      [0-2]             [3-5]
      sum=8             sum=16
     /    \            /      \
  [0-1]  [2-2]    [3-4]   [5-5]
  sum=7    1       sum=13    3
 /    \
[0]  [1]
2    5
```

Explain:

Each node stores **range information**.

Examples:

```
sum
min
max
gcd
```

---

4. Important Properties

Segment tree size:

```
4 × N
```

Height:

```
log2(N)
```

Time Complexity

Operation	Complexity
Build	O(N)
Query	O(log N)
Update	O(log N)

---

5. Build Segment Tree

Explain recursive build.

Core idea:

Split array into halves.

Build Algorithm

```
build(node,start,end)

if start == end
    tree[node] = arr[start]
else
    mid = (start + end)/2

    build(left child)
    build(right child)

    tree[node] = tree[left] + tree[right]
```

Explain:

```
node
2*node
2*node+1
```

Java Implementation

```
void build(int node, int start, int end) {

    if(start == end) {
        tree[node] = arr[start];
    }
    else {
        int mid = (start + end) / 2;

        build(2*node, start, mid);
        build(2*node+1, mid+1, end);

        tree[node] = tree[2*node] + tree[2*node+1];
    }
}
```

---

6. Range Query

Query example:

```
sum(1,4)
```

Explain **three cases**

Case 1: No overlap

```
return 0
```

Case 2: Complete overlap

```
return node value
```

Case 3: Partial overlap

Split into children.

Query Algorithm

```
query(node,start,end,L,R)

if R < start OR end < L
    return 0

if L <= start AND end <= R
    return tree[node]

mid = (start+end)/2
```

```
left = query(left child)
right = query(right child)

return left + right
```

Java Implementation

```
int query(int node, int start, int end, int L, int R) {

    if(R < start || end < L)
        return 0;

    if(L <= start && end <= R)
        return tree[node];

    int mid = (start + end) / 2;

    int left = query(2*node, start, mid, L, R);
    int right = query(2*node+1, mid+1, end, L, R);

    return left + right;
}
```

---

7. Update Operation

Example:

```
update index 3 → value 10
```

Steps:

1 Update leaf node
2 Update parent nodes

Update Algorithm

```
update(node,start,end,index,value)

if start == end
    arr[index] = value
    tree[node] = value
else
    mid = (start+end)/2

    if index <= mid
        update(left)
```

```
    else
        update(right)

    tree[node] = tree[left] + tree[right]
```

Java Implementation

```
void update(int node, int start, int end, int idx, int val) {

    if(start == end) {
        arr[idx] = val;
        tree[node] = val;
    }
    else {

        int mid = (start + end) / 2;

        if(idx <= mid)
            update(2*node, start, mid, idx, val);
        else
            update(2*node+1, mid+1, end, idx, val);

        tree[node] = tree[2*node] + tree[2*node+1];
    }
}
```

---

8. How Students Identify Segment Tree Problems

Tell students to look for these clues.

Pattern 1

Multiple queries

```
Q queries
```

Pattern 2

Range queries

```
sum(l,r)
min(l,r)
max(l,r)
```

Pattern 3

Updates present

```
update(i,val)
```

Pattern 4

Constraints

```
N ≤ 10^5
Q ≤ 10^5
```

Pattern 5

Range statistics

```
count
frequency
range maximum
```

---

9. Practice Problems (110 – 120 minutes)

Ask students to solve:

Easy

1. Range Sum Query Mutable
2. Range Minimum Query
3. Dynamic Range Sum

Medium

4. Count of Smaller Numbers After Self
5. Falling Squares
6. Range Module

Hard

7. Maximum Sum Queries
8. Reverse Pairs
9. Skyline Problem
10. Longest Increasing Subsequence II

Final Summary for Students

Segment Tree is used when:

```
Range Query + Updates
```

Operations supported:

```
Sum
Min
Max
GCD
XOR
Frequency
```

Complexity:

```
Build  → O(N)
Query  → O(log N)
Update → O(log N)
```

Homework for Students

Solve these problems:

```
LeetCode 307
LeetCode 315
LeetCode 327
LeetCode 699
LeetCode 715
```

Lazy Propagation in Segment Tree

By the end of the lecture students should be able to:

1. Understand why **Lazy Propagation** is required.
2. Identify problems requiring **range updates**.
3. Implement **range update + range query** efficiently.
4. Solve competitive programming problems using lazy propagation.

1. Motivation

Start with a simple array.

```
[2, 5, 1, 4, 9, 3]
```

Suppose queries are:

```
update(1,4,+3)
update(2,5,+2)
sum(1,3)
```

Meaning:

```
Add +3 to all elements between index 1 and 4
```

Result:

```
[2, 8, 4, 7, 12, 3]
```

Ask students:

How will you update the segment tree?

Without Lazy Propagation

You must update **each element individually**.

For update(1,4):

```
update index 1
update index 2
update index 3
update index 4
```

Time Complexity:

```
O(N log N)
```

Too slow for large inputs.

2. Idea of Lazy Propagation

Lazy Propagation means:

```
Delay the update until necessary
```

Instead of updating all nodes immediately:

```
Store update in a lazy array
```

Only apply it when required.

Concept:

```
tree[] → stores segment values
lazy[] → stores pending updates
```

---

3. Data Structures Used

Two arrays:

```
tree[4*N]
lazy[4*N]
```

Example:

```
tree[node] → segment sum
lazy[node] → pending update
```

---

4. How Lazy Propagation Works

When updating a range:

Instead of updating all children:

```
Mark node as lazy
```

Meaning:

```
This node still needs to propagate update to children
```

Update will be pushed later when needed.

---

5. Lazy Propagation Cases

Whenever visiting a node:

Case 1: Pending update exists

```
lazy[node] != 0
```

Then apply update to node first.

Apply Lazy Update

```
tree[node] += (end-start+1) * lazy[node]
```

If not leaf:

```
lazy[left child] += lazy[node]
lazy[right child] += lazy[node]
```

Then clear:

```
lazy[node] = 0
```

---

6. Range Update Algorithm

Range Update Example:

```
update(L,R,val)
```

Algorithm:

```
update(node,start,end,L,R,val)
```

Step 1

Apply pending lazy update.

Step 2

Check overlap cases.

No overlap

```
return
```

Complete overlap

Update node value.

Mark children lazy.

Partial overlap

Split and recurse.

Pseudocode

```
update(node,start,end,L,R,val)

apply pending lazy update

if R < start OR end < L
    return

if L <= start AND end <= R

    tree[node] += (end-start+1)*val

    if not leaf
        lazy[left] += val
        lazy[right] += val

    return

mid = (start+end)/2

update(left child)
update(right child)

tree[node] = tree[left] + tree[right]
```

---

7. Range Query with Lazy Propagation

Before answering query:

Check pending lazy updates.

```
if lazy[node] != 0
    apply update
```

Then proceed normally.

Query Pseudocode

```
query(node,start,end,L,R)

apply pending lazy update

if R < start OR end < L
    return 0

if L <= start AND end <= R
    return tree[node]

mid = (start+end)/2

left = query(left child)
right = query(right child)

return left + right
```

---

8. Java Implementation

Lazy Arrays

```
int tree[] = new int[4*N];
int lazy[] = new int[4*N];
```

Range Update

```
void update(int node, int start, int end, int l, int r, int val){

    if(lazy[node] != 0){
        tree[node] += (end-start+1)*lazy[node];
```

```
        if(start != end){
            lazy[2*node] += lazy[node];
            lazy[2*node+1] += lazy[node];
        }

        lazy[node] = 0;
    }

    if(r < start || end < l)
        return;

    if(l <= start && end <= r){

        tree[node] += (end-start+1)*val;

        if(start != end){
            lazy[2*node] += val;
            lazy[2*node+1] += val;
        }

        return;
    }

    int mid = (start+end)/2;

    update(2*node,start,mid,l,r,val);
    update(2*node+1,mid+1,end,l,r,val);

    tree[node] = tree[2*node] + tree[2*node+1];
}
```

Query

```
int query(int node,int start,int end,int l,int r){

    if(lazy[node] != 0){
        tree[node] += (end-start+1)*lazy[node];

        if(start != end){
            lazy[2*node] += lazy[node];
            lazy[2*node+1] += lazy[node];
        }

        lazy[node] = 0;
    }

    if(r < start || end < l)
        return 0;

    if(l <= start && end <= r)
        return tree[node];

    int mid = (start+end)/2;

    return query(2*node,start,mid,l,r)
```

```
        + query(2*node+1,mid+1,end,l,r);
}
```

---

9. Complexity

Operation	Complexity
Build	O(N)
Range Update	O(log N)
Range Query	O(log N)

Without lazy propagation:

```
Range update → O(N)
```

Huge improvement.

---

10. When Students Should Use Lazy Propagation

Look for these patterns:

Pattern 1

Range update problems.

Example:

```
add value to range
```

Pattern 2

Multiple updates + queries.

```
update(l,r,val)
query(l,r)
```

Pattern 3

Large constraints

```
N ≤ 10^5
Q ≤ 10^5
```

---

11. Important Practice Problems

Students should solve these.

Easy

1. Range Addition
2. Range Sum Query with Updates

Medium

3. Range Module
4. Falling Squares
5. Range Update Query

Hard

6. Maximum Sum Queries
7. Skyline Problem
8. Count of Range Sum
9. Reverse Pairs
10. Longest Increasing Subsequence II

---

12. Common Mistakes Students Make

1. Forgetting to clear lazy value.
2. Not applying lazy before query.
3. Wrong update formula.
4. Forgetting `(end-start+1)` multiplier.
5. Incorrect overlap conditions.